

Observability & Production Design

Digital Wealth Management Assistant — Monitoring, Reliability & Security Plan

1. Overview

This assistant handles informational queries (portfolio balance) and sensitive actions (fund transfers) gated behind identity verification. Each topic below covers **detection** (monitoring) and **prevention** (reliability/security design) together — two sides of one risk, not separate concerns. Four pillars: **Security, Reliability & Data Integrity, Agent Behavior, LLM Cost/Performance**. Two principles: (1) identity-verification failures and business-logic failures (e.g. insufficient funds) are tracked separately — conflating them blurs a security incident with routine noise; (2) metrics must never leak the sensitive values they measure.

2. Security: Monitoring & Prevention

Failed verification attempts — count of `validate_security_answer` returning `False`, per `user_id/session`. Alert: 3+ consecutive failures from one `user_id` in 10 min → manual review; 5+ from one session/IP across different `user_ids` → possible credential stuffing, page on-call. Dashboard: failed-attempt rate by `user_id`, to separate "one confused customer" from systematic probing.

Unauthorized transfer attempts (detect + prevent) — metric: `execute_transfer` attempted while state $\neq$ `VERIFIED`; should be near-zero, so any occurrence pages on-call as the top security signal (logic bypass or prompt-injection jailbreak). Prevention: this check originally lived only inside `execute_transfer` — a future sensitive tool could forget it. Fixed with `security_gate_before_tool`, registered as ADK's `before_tool_callback`, running before any tool executes and independently blocking unverified calls in `SENSITIVE_TOOLS` before the real function ever runs — deliberately redundant with `execute_transfer`'s own gate, so no single point of failure.

State inconsistencies — sessions where verification transitions violate the expected graph (e.g. `VERIFIED` with no prior `QUESTION_ISSUED`). Detect via a structured event per transition (`session_id`, from/to state, timestamp) validated against the allowed set. Any invalid transition routes to engineering as a data-integrity bug, not to fraud/security ops.

Prompt injection & sensitive-data exposure — the transfer-approval decision is enforced in Python, not model judgment, so a jailbreak that convinces the model to *attempt* the call is still refused by the two gates above. Not modeled: indirect injection via tool *output* (low risk today since data is fixed/seeded). Tool responses are replayed to Gemini every turn, so exact balances and the security question text do reach Google's API as plaintext — the one value never exposed this way is the *correct* stored `security_answer`, compared only in Python. Our own logs/telemetry carry the same risk: never log raw answers or balances — emit booleans/counts only (`answer_correct: false`), even in the high-retention security stream in Section 6.

3. Reliability & Data Integrity of the Transfer Operation

Concurrency — found and fixed. `execute_transfer` originally did `SELECT` → compute in Python → `UPDATE`, an unguarded read-modify-write: two concurrent transfers for one `user_id` could read the same balance and one would silently overwrite the other. Fixed with a single atomic `UPDATE ... WHERE balance >= ?`, checked via `cursor.rowcount` — debit, credit, and the funds check all execute as one indivisible DB operation. Added `PRAGMA busy_timeout` so writers wait instead of failing on contention. Verified with a test firing 5 concurrent transfers at one account, asserting all 5 land — this fails under the old code.

Idempotency & crash consistency — proposed, not implemented. A crash between verification and transfer request is safe (nothing happened yet). The real gap: a crash *after* `commit()` but *before* the caller learns the result — money moved, but a retry would move it again. No idempotency key or transfer ledger exists today (only a running balance). Proposed fix: a session-derived idempotency key checked against a `transfers` ledger before applying any debit/credit — standard for payment APIs, not built here to keep scope to the assignment.

Session-store durability — proposed, not implemented. State survives a crash only if the backing store does: `adk web` persists to real SQLite (`wealth_agent/.adk/session.db`); `pytest / adk eval` use an in-memory store that evaporates on exit. A real deployment needs a durable, replicated session store with a stated crash guarantee — this project inherits ADK's dev-only default.

4. Agent Metrics

Tool execution failures — error rate per tool, tagged by type (DB exception, timeout, malformed LLM args). A spike isolated to one tool usually means a schema/DB issue; across all tools, an infra problem.

Loop detection — tool calls per session before completion; repeated identical calls with identical args. The 3-attempt verification cap doubles as loop prevention for that flow; anything exceeding it is force-terminated and logged, not silently retried.

Latency — TTFT per turn (illustrative target p95 < 2s); multi-step workflow latency, request-to-final-result (p95 < 8s for a full verify-and-transfer flow). Track p50/p95/p99, not averages — a small number of slow outliers (DB lock contention) matter disproportionately here. Targets are starting points, not fixed.

5. LLM Metrics

Token usage — per turn, aggregated per session/user, input vs. output split — the ratio differs a lot between a one-shot balance query and a full transfer flow.

Cost tracking — $\text{cost} = \text{input_tokens} \times \text{input_rate} + \text{output_tokens} \times \text{output_rate}$, per session, rolled up daily/monthly, segmented by intent type. Cross-linked with loop detection: a 10x cost spike on one session is very likely also a stuck-tool-calling session — one anomaly, both dashboards.

API quota/rate-limit exhaustion — count of 429 s, tagged per-minute vs. per-day (different fixes: backoff vs. model/key fallback). Not hypothetical — this project hit a 20/day free-tier cap mid-testing (see `AI_USAGE.md`). Any per-day exhaustion pages the API-key/billing owner — it's a full outage for every user on that key.

6. Implementation Notes (not built for this exercise, but design-ready)

- **Correlation:** every metric above should share one key per turn. ADK already generates an `invocation_id` internally (confirmed while building this project) — the natural trace ID, one span per LLM/tool call, so "why slow" and "why blocked" are both answerable from one trace.
- Events emitted as structured logs/traces (e.g. OpenTelemetry spans), not free text, so they're queryable and alertable.
- Security-relevant events route to a separate, higher-retention, access-controlled stream (subject to the redaction rule in Section 2).
- Dashboards split by audience — security/fraud (§2), engineering reliability (§3–4), cost/finance (§5) — one shared dashboard buries the signal each stakeholder needs.